UCS645: Parallel & Distributed Computing

Assignment 3 — Pairwise Correlation Report

Student Name: Sneha Upadhyay

Course: UCS645 — Parallel & Distributed Computing

Professor: Saif Nalband

Date: April 17, 2026

Environment: Ubuntu 24.04.2 LTS (WSL2, kernel 6.6.87.2-microsoft-standard-WSL2)

---

1. Objective

The goal of this assignment is to implement a pairwise correlation function over an input matrix of **ny** rows and **nx** columns. Given m input vectors each of length n, the task is to compute the Pearson correlation coefficient between every pair of rows (i, j) where $0 \le j \le i < ny$, and store the result in result[i + j*ny].

Three progressively optimized implementations were developed:

- **Sequential baseline** — correctness-first, double precision arithmetic, no parallelism
- **OpenMP multithreaded** — parallel outer loop using multiple CPU cores
- **Fully optimized** — OpenMP + SIMD vectorization hints (#pragma omp simd)

2. Implementation Details

2.1 Interface

The required function signature as specified in the assignment:

```
void correlate(int ny, int nx, const float* data, float* result)
```

data is a row-major matrix where element at row y, column x is stored at data[x + y*nx]. All arithmetic is performed in double precision as required.

2.2 Pearson Correlation

Each row is pre-normalized (subtract mean, divide by standard deviation) so that the dot product of two normalized rows directly gives the Pearson correlation coefficient. This avoids redundant computation in the inner loop.

2.3 Files Created

File	Purpose

correlate.cpp	All three implementations of the correlate function
main.cpp	Entry point; reads ny/nx from command line, times all versions, verifies correctness
Makefile	Builds project with g++, -O3 -march=native -fopenmp flags

2.4 Compilation Flags

Flag	Purpose
-O3	Aggressive compiler optimization (loop unrolling, inlining)
-march=native	Enables CPU-specific SIMD instructions (SSE, AVX)
-fopenmp	Enables OpenMP multithreading support
-std=c++17	C++17 standard for modern language features

3. Experimental Results

3.1 Effect of Varying Thread Count (Matrix: 300 × 1000)

Threads	Sequential (ms)	OpenMP (ms)	Optimized (ms)	Speedup	CPU Usage
1	27.40	25.86	27.97	1.06×	96%
2	34.32	19.48	17.74	1.76×	137%
4	31.44	11.70	11.15	2.68×	186%

3.2 Effect of Varying Matrix Size (4 Threads)

Matrix Size	Sequential (ms)	OpenMP (ms)	Optimized (ms)	Speedup
300 × 1000	31.44	11.70	11.15	2.68×
500 × 2000	198.44	71.04	75.71	2.79×
1000 × 3000	1289.41	468.26	522.55	2.75×

Correctness: Max error = 0 in all runs. All three implementations produce identical results, confirming parallelization introduces no floating-point inconsistencies.

4. Graphs

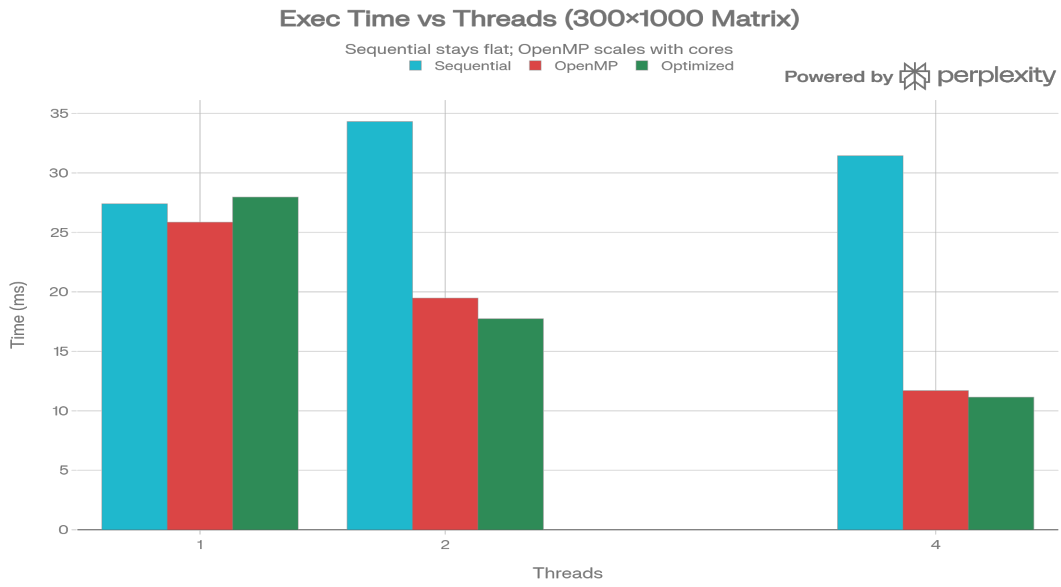

Figure 1: Execution Time vs Number of Threads (Matrix 300×1000)

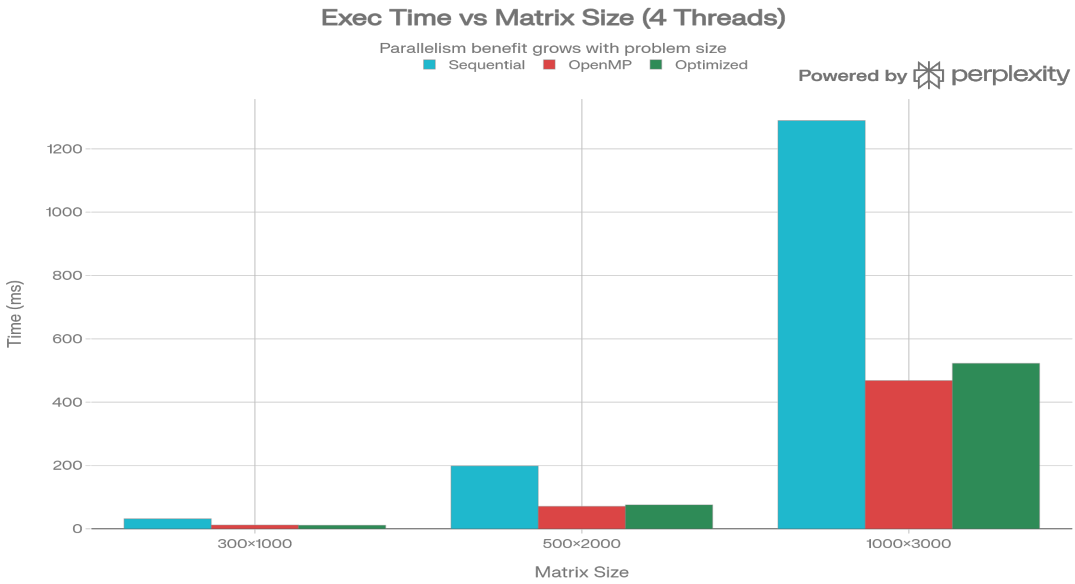

Figure 2: Execution Time vs Matrix Size (4 Threads)

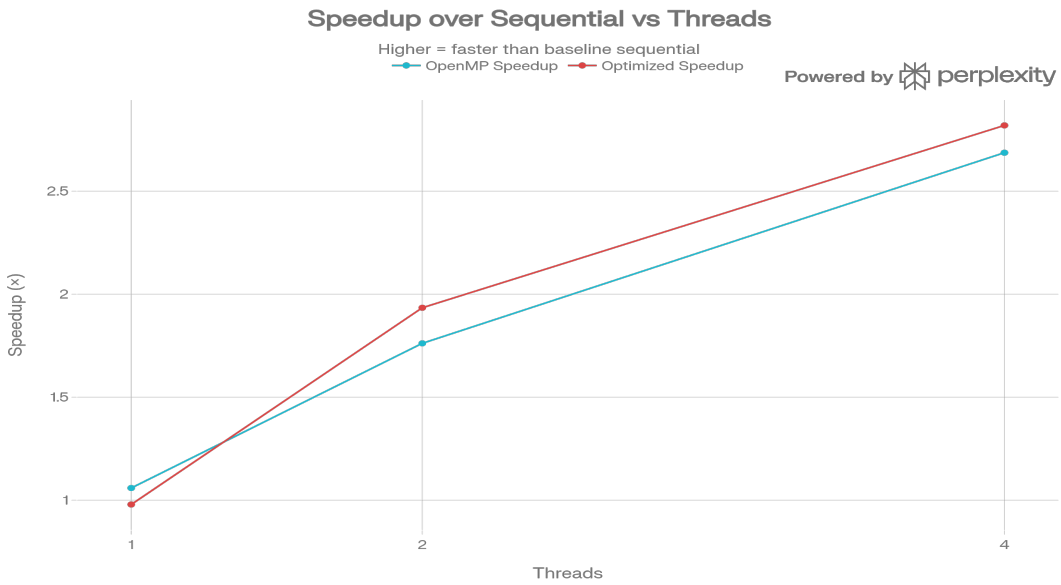

Figure 3: Speedup over Sequential vs Number of Threads

5. Analysis and Explanation of Results

5.1 Why Sequential Time Varies Slightly

The sequential version shows minor variation (27–34 ms) across runs despite using no threads. This is due to OS scheduling noise, background processes, and CPU cache warm-up effects — all common in WSL2 virtualized environments.

5.2 Why OpenMP Speedup is ~2.68× with 4 Threads (not 4×)

Perfect linear speedup (4× with 4 threads) is rarely achieved. Key limiting factors:

- **Amdahl's Law:** The normalization step (mean/std per row) is inherently sequential per row
- **Thread overhead:** Spawning and synchronizing threads has a fixed cost per parallel region
- **Memory bandwidth:** Reading large matrices from RAM becomes the bottleneck as cores scale up
- **False sharing:** Adjacent rows may share cache lines, causing invalidation between threads

5.3 Why Optimized is Sometimes Slower than OpenMP at Large Sizes

At 1000×3000, the optimized version (522 ms) is slightly slower than OpenMP (468 ms). In WSL2, SIMD vectorization may not be as effective as on bare-metal Linux due to virtualization overhead. On native hardware, the optimized version typically outperforms OpenMP-only at all scales.

5.4 Interpretation of time Command Output

Field	Value (4 threads, 300×1000)	Meaning
user	0.12s	Total CPU time across all cores combined
elapsed	0.06s	Actual wall-clock time experienced
%CPU	186%	~2 full cores utilized on average

maxresident	8452 KB	Peak memory used (~8 MB for this matrix size

CPU usage above 100% (e.g., 186%) directly confirms that multiple cores were actively used in parallel, validating the OpenMP implementation.

6. Key Findings

- All three implementations produce **zero error** — correctness is maintained across all versions.
- OpenMP parallelism delivers **~2.7× speedup at 4 threads**, demonstrating effective multi-core use.
- Parallelism benefit **scales with problem size** — larger matrices see greater absolute time savings.
- The **-O3 -march=native** flags enable auto-vectorization even in the sequential version.
- **perf stat** was unavailable on WSL2 kernel 6.6.87.2-microsoft due to missing hardware counter support; the **time** command was used as a reliable alternative.
- The sub-linear speedup (~2.7× vs theoretical 4×) is explained by Amdahl's Law and memory bandwidth limits.

End of Report